DevSys VM — OrbStack Golden-Clone Runbook

Identical, version-pinned Ubuntu dev VMs on a Mac with OrbStack — build one golden base machine, clone it per developer in seconds, reach each clone over Tailscale.

The system: three scripts in the `devsys-orbstack` folder. `build-base.sh` and `newvm.sh` run on the Mac; `devsys-vm.sh` (installer 3.2.0) runs inside the guest. You install the pinned toolchain once into a base machine called `devsys-base`, and every developer VM is a clone of it. Each clone joins Tailscale tagged `tag:devsys` + `tag:<dev>`; the admin reaches every VM, each developer reaches only their own.

Phase	What	How often	Where
0	Prepare the Mac (OrbStack, the scripts)	once	the Mac
1	Tailscale tags, SSH rules, OAuth client	once, plus one tag and rule per developer	Tailscale admin console
2	Build the golden base	once, and again to change tool versions	the Mac
3	Clone a developer VM	per developer	the Mac
4	Verify and operate	ongoing	the Mac / inside a VM

Phase 0 — Prepare the Mac (one-time)

0.1 Install OrbStack

Install OrbStack (`brew install orbstack`, or the download from orbstack.dev) and open it once so that `orbctl` is on the PATH. Check both tools the scripts need:

```
orbctl version
jq --version
```

The VMs run only while OrbStack is running. Set OrbStack to start at login and keep the Mac from sleeping, or developers lose their VMs when it does.

0.2 Get the scripts

```
git clone https://github.com/sohebcakewalk/dev-setup.git
cd dev-setup/devsys-vm/devsys-orbstack
```

0.3 Tailscale on the Mac (optional)

Install the Tailscale app and sign in as a tailnet admin if you want to `ssh` into the VMs from this Mac. Without it you can still get a root shell in any VM with `orbctl run`.

Phase 1 — Tailscale setup (one-time, tailnet-level)

All in the Tailscale admin console. The tags and rules are the same ones the Incus setup uses.

1.1 Tags (Definitions → Tags)

Tag	Owner
`tag:devsys`	`autogroup:admin, tag:devsys`
`tag:soheb`	`tag:devsys`
(one per future dev)	`tag:devsys`

`tag:devsys` must own itself — a key carrying two tags needs every tag, including `tag:devsys`, to pass the ownership check. Without it `newvm.sh` stops with "Auth key mint failed".

1.2 Tailscale SSH rules (Access controls → Tailscale SSH)

#	Sources	Destination	Users
1	`autogroup:member`	`autogroup:self`	`autogroup:nonroot`, `root`
2	`autogroup:admin`	`tag:devsys`	`root`
3	`soheb@example.com`	`tag:soheb`	`soheb`
+1 per dev	`<dev>@example.com`	`tag:<dev>`	`<dev>`

Rule 2 is the admin's root reach across every VM; rules 3 and up let each developer into only their own VM. Never use `tag:devsys` as a source — it would let every VM reach every other VM.

1.3 OAuth client, stored on the Mac

Settings → OAuth clients → scope Auth Keys: Write, Tags = `tag:devsys` plus every person-tag. `newvm.sh` uses it to mint one auth key per VM. Store the client ID and secret in the macOS Keychain:

```
security add-generic-password -s ts-oauth -a tailscaleclientid -w '<id>'
security add-generic-password -s ts-oauth -a tailscaleclientsecret -w '<secret>'
```

`newvm.sh` looks for the client in this order: the environment (`DEVSYS_TS_CLIENT_ID`, `DEVSYS_TS_CLIENT_SECRET`), then the mode-600 file `~/.config/devsys/ts-oauth.env` defining the same two variables, then the Keychain items above.

Phase 2 — Build the golden base (one-time)

```
./build-base.sh
```

One command does the whole phase:

1. Creates the machine `devsys-base` from `ubuntu:noble`, with `--isolated --isolate-network` so the Mac's home folder, the other machines' files and machine-to-machine networking are all hidden. Clones inherit this.
2. Copies the `devsys-vm.sh` beside it into the machine and installs every group — no developer user, no Tailscale login.
3. Clears the per-machine identity (machine-id, Tailscale state, SSH host keys).
4. Stops the machine.

On an M1 Pro this took about 3 minutes and 3.4 GB. Check the result with `orbctl list`: `devsys-base` must show as stopped.

The stopped base is the image. Do not start it by hand: a running base regenerates the identity that clones must not share, and `newvm.sh` refuses to clone it. To change it, re-run `./build-base.sh`.

Option (environment variable)	Effect
`DEVSYS_GROUPS="node python ai"`	install only these groups into the base
`DEVSYS_SHARED_CLAUDE=1`	bake in the shared-Claude setup (several Unix users, one Claude login)
`DEVSYS_BASE_IMAGE=ubuntu:noble`	distro image for a new base
`./build-base.sh devsys-base-next`	build under a different base name

Phase 3 — Clone a developer VM (per developer)

First, in the Tailscale admin console, give the developer their tag and SSH rule (Phase 1.1 and 1.2) and add the new tag to the OAuth client. Then, on the Mac:

```
./newvm.sh sohebvm soheb
```

The first argument is the VM name (lowercase letters, digits, hyphens), the second the developer's Linux username. The script:

1. Checks that the base exists and is stopped, and that no machine has that name.
2. Mints a Tailscale auth key tagged `tag:devsys` + `tag:soheb`: single-use, not ephemeral, pre-approved, valid for one hour.
3. Clones `devsys-base` and starts the clone.
4. Inside the clone, creates the user and joins Tailscale with SSH enabled. The key is passed on stdin and is never written to a file or a command line.

Flag	Effect
`--login-link`	no admin key: prints a `login.tailscale.com/a/…` link to send to the developer, who must already be a tailnet member and becomes the node's owner
`--base NAME`	clone from a different base machine

If setup inside the clone fails, the VM is left in place. Inspect it with `orbctl run -m <vm> -u root bash`, or remove it with `orbctl delete <vm>` and run `newvm.sh` again.

Phase 4 — Verify and operate

Check / task	Command
List machines and their state	`orbctl list`
Admin reach (rule 2)	`ssh root@sohebvm`
Dev login (rule 3)	`ssh soheb@sohebvm`
Isolation (must fail)	`ssh soheb@`<another dev's VM> → denied
Health and versions	`orbctl run -m sohebvm -u root devsys status`
Break-glass root shell	`orbctl run -m sohebvm -u root bash`
Change tool versions	edit the `MISE_*`/`NPM_*` pins in `devsys-vm.sh` → `./build-base.sh` → new clones get them; existing clones are not touched
Retire a VM	`orbctl delete sohebvm`, then remove it from Tailscale → Machines

Do not run `devsys update` inside a VM yet. It downloads the published gist copy, which is still 3.1.6, and replaces 3.2.0 with it. That is safe again once `devsys-vm.sh` is republished to the gist as `devsys-v3.sh`.

When something stops

Message	Fix
`orbctl not found`	install OrbStack and open it once (Phase 0.1)
`No base machine 'devsys-base'`	run `./build-base.sh`
`Base 'devsys-base' is running, not stopped`	re-run `./build-base.sh`; do not stop it by hand, its identity has to be cleared again
`A machine named '…' already exists`	pick another name, or `orbctl delete` the old one
`No Tailscale OAuth client`	store the client ID and secret (Phase 1.3)
`OAuth token exchange failed`	the stored ID or secret is wrong or revoked
`Auth key mint failed`	`tag:<dev>` is missing, is not owned by `tag:devsys`, is not on the OAuth client, or `tag:devsys` does not own itself (Phase 1.1)
“not permit you to SSH as user X”	the per-developer SSH rule is missing (Phase 1.2)

Moving a developer over from the Incus server

An Incus VM cannot be imported into OrbStack. Build the VM fresh with `newvm.sh` under a different Tailscale name, then copy the home folder through an admin machine that reaches both VMs as root:

```
EXCLUDES='--exclude=node_modules --exclude=.venv --exclude=.cache'
ssh root@<old-vm> "tar -C /home/soheb $EXCLUDES -czf - ." \
  | ssh root@<new-vm> 'tar -C /home/soheb -xzpf -'
```

Reinstall `node_modules` and `.venv` in each repo afterwards. Do not copy `/opt/devsys/mise`, `/var/lib/tailscale` or `/etc/machine-id`. Once the old VM is retired, rename the new one inside the VM with `tailscale set --hostname=<name>`. This recipe has not been run against the Incus server.

Tested on an Apple Silicon Mac (M1 Pro, OrbStack 2.2.4): `build-base.sh` built a base with all 12 groups; `newvm.sh --login-link` cloned it in 6 seconds with its own hostname and machine-id, the dev user, no `/mnt/mac`, and every tool at its pinned version.

Not yet tested: the Mac Studio itself; key mode (minting a real Tailscale key and joining the tailnet — that code is carried over from the Incus `newvm.sh`); `DEVSYS_SHARED_CLAUDE=1` on OrbStack; the migration recipe.